

Computer Security

Data Integrity

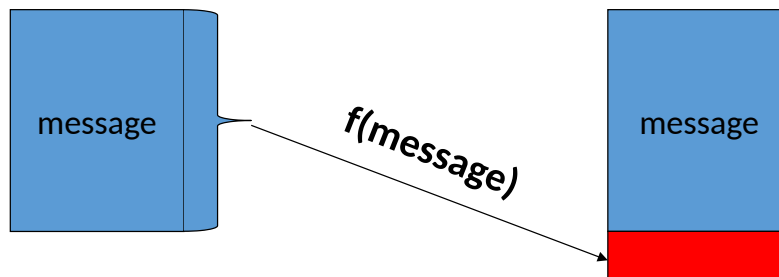
Guellil zouaoui

Objectives of Computer Security (Review)

- The classic **CIA Triad (Confidentiality, Integrity, Availability)**
 - **Confidentiality:** Ensuring information remains secret and is accessible only to authorized entities.
 - **Integrity:** Guaranteeing that data has not been altered in an unauthorized manner during transmission or storage.
 - **Availability:** Ensuring that systems and data are accessible to authorized users when needed.
- **Authentication:** Verifying the identity of an entity (user, system, or device) with a high degree of certainty.
- **Non-Repudiation:** Providing proof that a specific action was performed by a specific entity, preventing that entity from later denying its involvement.

Digital Signature

- The International Organization for Standardization **ISO/IEC 7498-2** formally defines a digital signature as:
- *"Data appended to, or a cryptographic transformation of, a data unit that allows a recipient of the data unit to prove the source and integrity of the data unit and protect against forgery."*



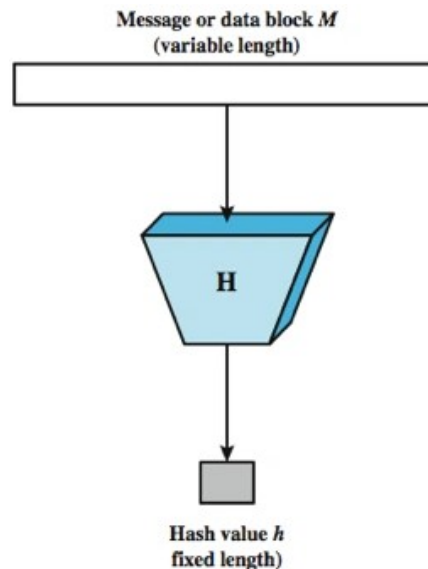
P.S.: $f()$ is a digital signature algorithm.

Digital Signatures: Overview

- A digital signature replicates the key characteristics of a handwritten signature:
 - **Binds a document to its author** (authentication)
 - **Makes forgery difficult** (integrity and non-repudiation)
- Digital signatures rely on **two complementary families of algorithms**:
 - **Asymmetric Encryption**(Public-Key) : Provides authentication and non-repudiation through private/public key pairs
 - **Hash Functions**: Ensures message integrity by generating a unique fingerprint (digest) of the data

Hash Functions

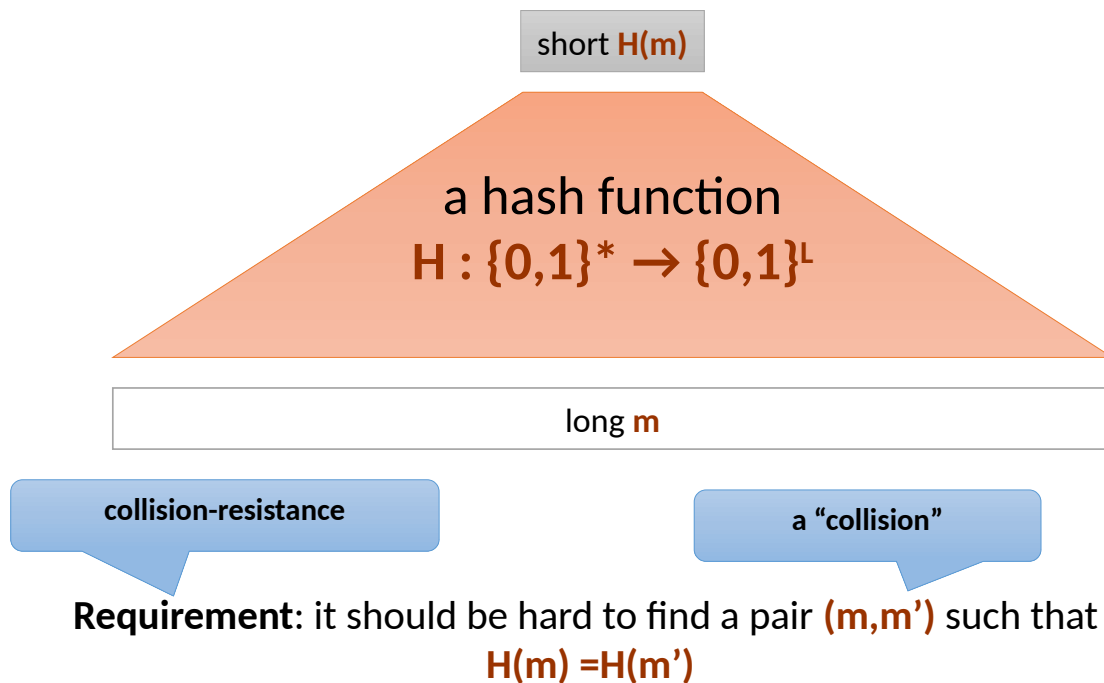
- **Definition :** A **hash function** is a one-way, collision-resistant function that produces a fixed-size output (called a **digest**, **hash value**, or **fingerprint**) that uniquely characterizes the input data.



Hash Functions: Properties

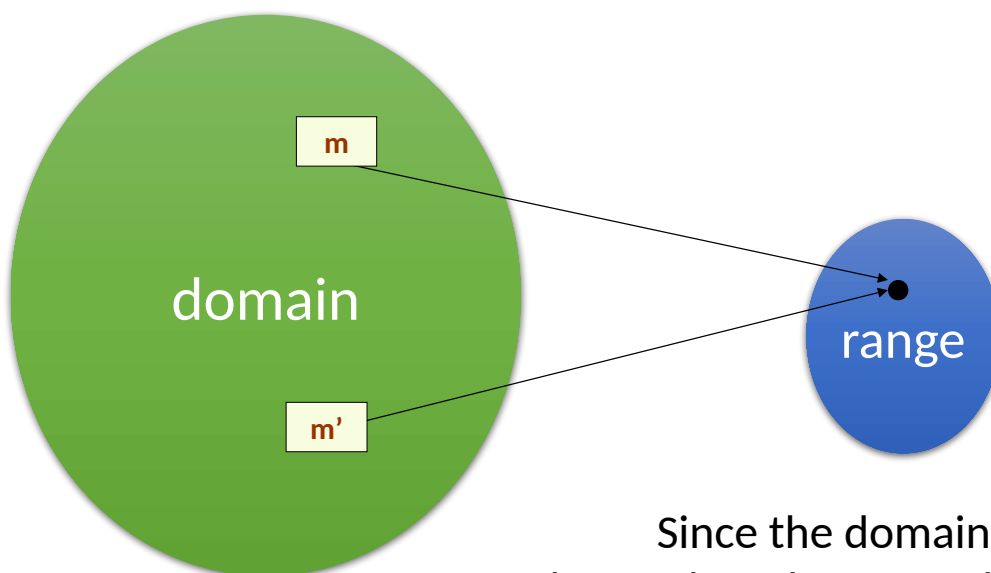
- A cryptographic hash function H must satisfy the following properties:
- 1. **Variable Input Size :** Can be applied to data blocks of **any size** — from a single byte to multi-gigabyte files.
- 2. **Fixed Output Length :** Always produces a digest of **fixed length** (e.g., 256 bits for SHA-256), regardless of input size.
- 3. **Efficiency :** For any input x , computing $H(x)$ must be **fast and practical**.
- 4. **Preimage Resistance (One-Way Property):** Given a hash value h , it is **computationally infeasible** to find any input x such that $H(x) = h$.
- 5. **Second Preimage Resistance:** Given an input x , it is **computationally infeasible** to find a different input $y \neq x$ such that $H(y) = H(x)$.
- 6. **Collision Resistance :** It is **computationally infeasible** to find **any two distinct inputs** $x \neq y$ such that $H(x) = H(y)$.

Collision-resistant hash functions



7

Collisions always exist



Since the domain is larger than the range the collisions have to exist.

8

Simple Hash Function

	bit 1	bit 2	• • •	bit n
block 1	b_{11}	b_{21}		b_{n1}
block 2	b_{12}	b_{22}		b_{n2}
	•	•	•	•
	•	•	•	•
	•	•	•	•
block m	b_{1m}	b_{2m}		b_{nm}
hash code	C_1	C_2		C_n

Figure 3.3 Simple Hash Function Using Bitwise XOR

Block1 Xor Block2 Xor Block3 Xor..... Blockn = hash code

Examples of Hash Functions

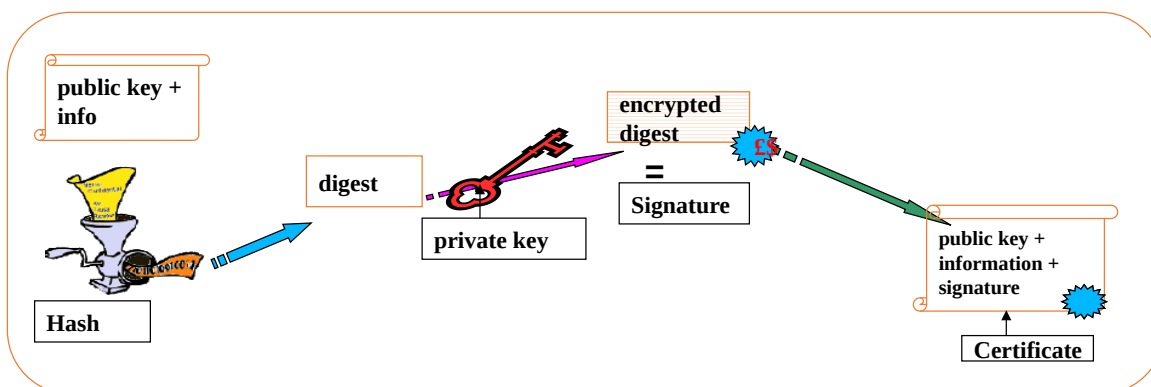
- **MD5 (Message Digest 5)** : 128-bit fingerprint (output).
Note: Now considered broken due to collision attacks.
- **SHA (Secure Hash Algorithm)** : NIST standard. 160-bit fingerprint.
- **SHA-1** (revised version published in 1994) : Considered more secure than MD5.
Note: Now considered weak; NIST deprecated it in 2011.
- **SHA-2** (October 2000) : Increased fingerprint size (224, 256, 384, or 512 bits).
Note: Still widely used and considered secure (e.g., SHA-256).
- **SHA-3** (NIST standard, October 2012) : Based on a completely different principle than MD5, SHA-1, and SHA-2, following the discovery of possible attacks against those earlier functions.

Merkle-Damgård Hash Construction

- A design framework for building cryptographic hash functions that:
- Uses **iterative compression** of data blocks (e.g., 512-bit blocks for **MD5**, **SHA-1**/SHA-2)
- **How It Works**
 1. **Padding:** Input is padded to a multiple of the block size (e.g., add 1, then 0s, then message length).
 2. **Block Processing:** Split padded input into fixed-size blocks $M_1, M_2, \dots, M_n$.
 3. **Compression Function (f)** :Iteratively processes each block using,
 1. **Initial Value (IV):** A fixed starting state
 2. **Chaining Variable:** The output of one step becomes the input for the next
 4. **The Iterative Chain:** $H_0 = IV, H_i = f(H_{i-1}, M_i)$
 5. **Final Hash:** The result H_n after processing the final block (and sometimes an optional final transformation).

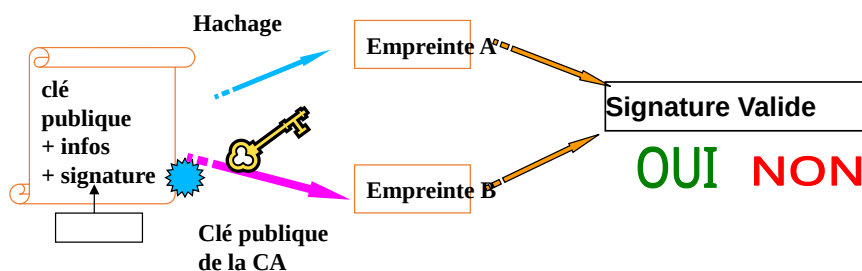
Document Signature

- A signature is computed by a cryptographic algorithm and bound to the data in such a way that recipients can verify that:
 - The data has **not been altered** (integrity)
 - The data **genuinely comes from the sender** (authentication)
- **Hash function** (e.g., MD5, SHA-1, SHA-256): Computes a fixed-size **digest** (or fingerprint) that summarizes the message.
- **Asymmetric encryption** (e.g., RSA, DSA, ECDSA)
 - The digest is encrypted using the **sender's private key**.
 - The result is the **digital signature** of the message.

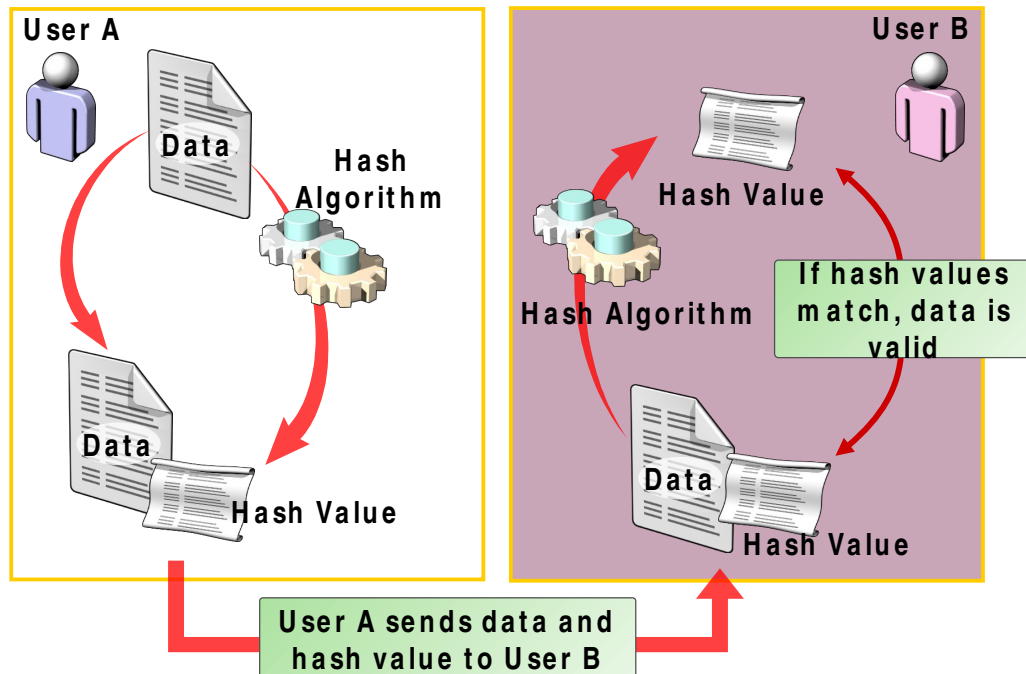


Signature Verification Process

- When a receiver gets a message, they perform the following steps:
- **Decrypt the signature** using the sender's **public key** to obtain the reference digest (original fingerprint).
- **Compute the digest** of the received message using the same hash function to obtain a **candidate digest**.
- **Compare** the two digests:
 - If they are **identical** → the message is **valid** (authentic and unaltered).
 - If they differ → the message has been **tampered with** or the signature is **forged**.



Verifying Simple Data Integrity with Hashes (Message Digests)



Message Authentication Code (MAC)

- The concept is relatively similar to hash functions. These are algorithms that create a small **authenticator block** of fixed size.
- The major difference is that this authenticator block is **no longer based solely on the message**, but also on a **secret key**.
- **Use of Conventional (Symmetric) Encryption:** Only the **sender** and **receiver** share the secret key.
- **Authentication Without Message Encryption:** An **authentication field** is generated and appended to each message.
- **MAC Definition:** A **MAC** is computed as a function of the message M and the secret key K : $MAC = F(K, M)$

Message Authentication Codes

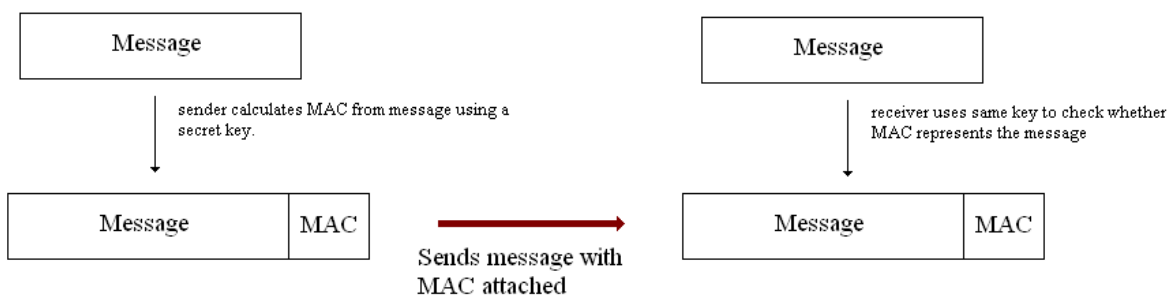
- Condenses message into a short hash



- For example, encrypt only the MAC with a key known to sender and receiver.

Message Authentication Code

- Alternatively, use a secret key.
- This also provides authentication.



MAC vs. Digital Signature (2/2)

- MAC (Message Authentication Code) and digital signatures are two mechanisms used to ensure **data integrity** and **authenticity**, but they differ in several ways.

1. Keys Used

- **MAC:** Uses a **shared symmetric secret key** between sender and receiver to generate and verify the authentication code.
- **Digital Signature:** Uses an **asymmetric key pair**. The sender signs the message with their **private key**, and the recipient verifies the signature using the sender's corresponding **public key**.

2. Non-repudiation

- **MAC:** Provides **no non-repudiation** because the same key is used to generate and verify the MAC. If a dispute arises, the sender could deny sending the message.
- **Digital Signature:** Provides **non-repudiation** because it uses the sender's private key to sign the message. The sender cannot deny having sent the message.

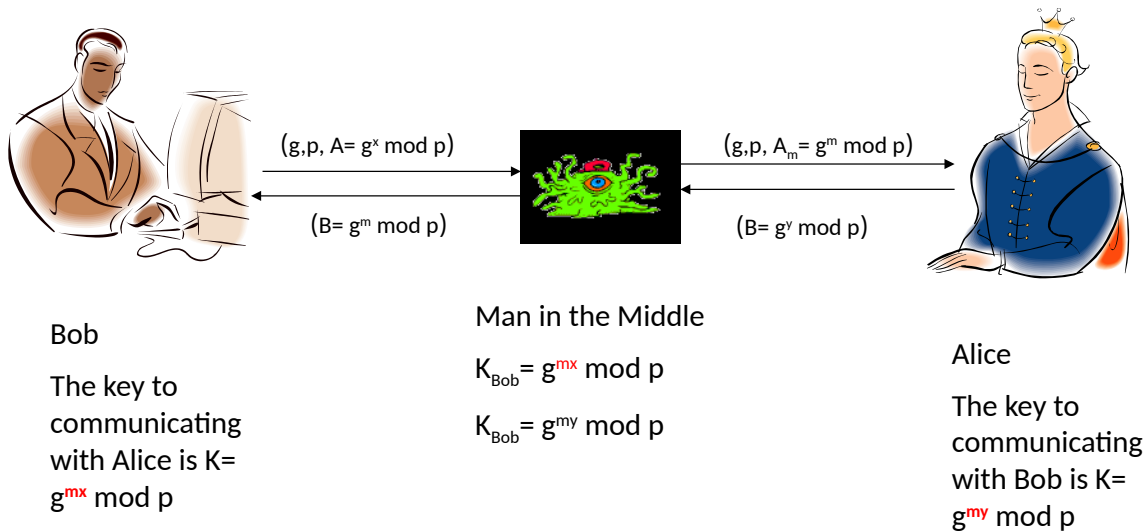
MAC vs. Digital Signature (2/2)

3. Usage Scenarios

- **MAC:**
 - Used in environments where parties **mutually trust** each other and share a secret key (e.g., **VPNs**, secure internal communications within an organization).
 - Ideal when parties communicate frequently and can securely share a secret key.
- **Digital Signature with Asymmetric Cryptography:**
 - Used in environments requiring **public authentication** (e.g., online transactions, digital certificates, secure communications between parties that **do not trust each other**).
 - Commonly applied to **one-time transactions** or scenarios where parties cannot pre-share keys.

Diffie Hellman (review)

- Man in the middle attack



Diffie Hellman (review)

- Problems with Basic Key Exchange:
 - Interlocutors are not authenticated → A man-in-the-middle (MitM) attack is always possible.
 - Exchanges lack integrity → An attacker can modify the shared secret during exchange.
- Solution:
 - New authenticated key exchange schemes are used.
 - An infrastructure to authenticate public keys enables secure exchange of session keys.



Bob

The private key : K_{Bs}

Public key : K_{Bp}

Bob, $g, p, B = g^x \bmod p$

Alice, $A = g^y \bmod p, \text{sign}_{K_{As}}(B = g^x, Y = g^y)$

Bob, $\text{sign}_{K_{Bs}}(B = g^x, Y = g^y)$



Alice

The private key : K_{As}

Public key : K_{Ap}

- **Prevents man-in-the-middle attacks:** It is impossible for an attacker to provide $\text{SIG}_{K_{Bs}}(B = g^x, A = g^y)$.
- **Prevents replay attacks:** The Diffie-Hellman value of the correspondent is also signed.

Identity Theft



Bob

The private key : K_{Bs}

Public key : K_{Bp}

Bob, $g, p, B = g^x \bmod p$

Alice, $A = g^y \bmod p, \text{sign}_{K_{As}}(g^x, g^y)$

Bob, $\text{sign}_{K_{Bs}}(g^y, g^x)$



Man in the Middle (MiM)

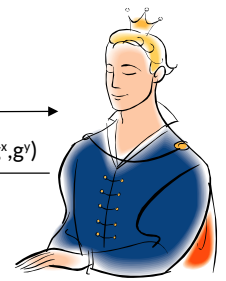
The private key : K_{MiMs}

Public key : K_{MiMp}

MiM, $g, p, B = g^x \bmod p$

Alice, $A = g^y \bmod p, \text{sign}_{K_{As}}(g^x, g^y)$

MiM, $\text{sign}_{K_{MiMs}}(g^y, g^x)$



Alice

The private key : K_{As}

Public key : K_{Ap}

- Man in the middle attack

SIGMA Protocol (SIGn and MAc)

- The SIGMA protocol (SIGn and MAc) was proposed by **Hugo Krawczyk** at *CRYPTO 2003*.
- SIGMA protocols provide **perfect forward secrecy (PFS)** through an **authenticated Diffie-Hellman key exchange** combined with **digital signatures**.
- These protocols are highly suitable for **real-world applications** and **standardized key exchange**.
- In particular, SIGMA serves as the **cryptographic foundation** for the signature-based modes of the **Internet Key Exchange (IKE)** protocol (versions 1 and 2).

SIGMA Scheme (SIGn and MAc) – Protocol Phases

- The SIGMA protocol consists of three phases:
- **Phase 1: Key Exchange:** Alice and Bob perform a **Diffie-Hellman key exchange** to generate a shared secret key.
- **Phase 2: Authentication:** Alice and Bob **authenticate each other** using their **digital signatures**. They sign the shared secret key generated in the previous phase and send the signatures to each other.
- **Phase 3: Key Confirmation:** Alice and Bob use **MAC** to confirm the authenticity of the shared secret key. They generate a MAC using the shared secret key and send it to each other.

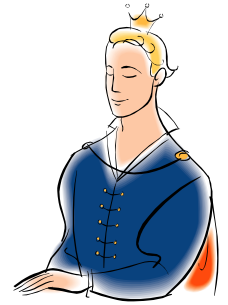
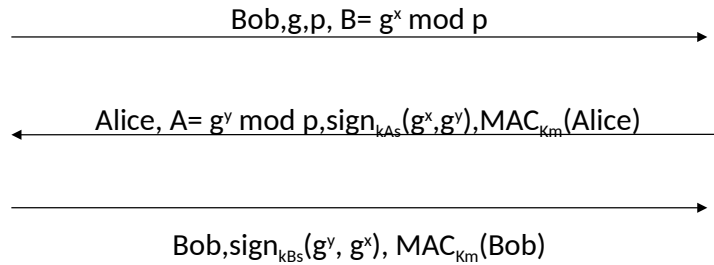
SIGMA Scheme (SIGn and MAc)



Bob

La clé privé : K_{Bs}

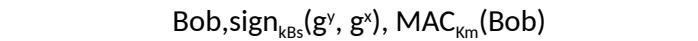
Clé publique : K_{Bp}



Alice

La clé privé : K_{As}

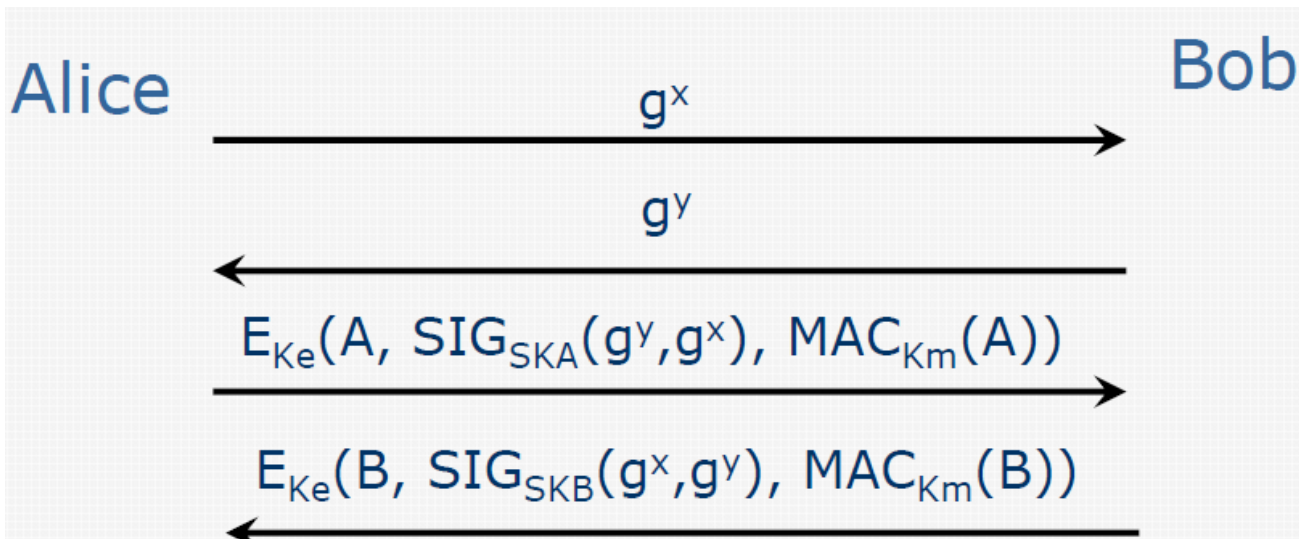
Clé publique : K_{Ap}



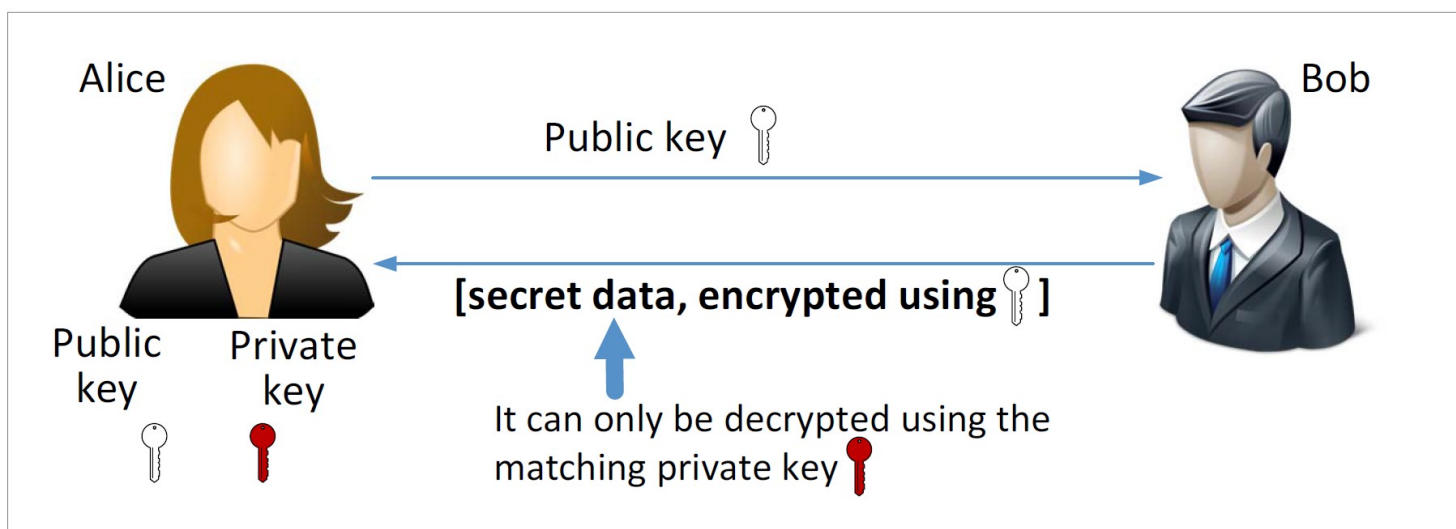
The MAC key K_m and the shared session key K_s are derived from g^{xy} , using a pseudorandom function

SIGMA with identity protection

- Encrypting the last two exchanges ensures that identities are protected,
- Encryption E must be secure against active attacks,

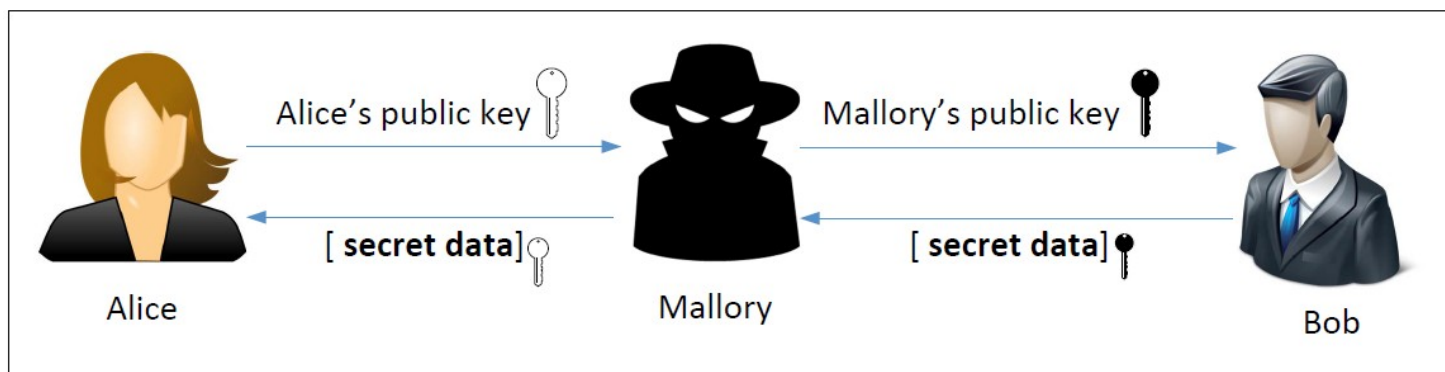


Review: Asymmetric Cryptography



29

ManInTheMiddle (MITM) Attack



Bob is in a tricky situation because he can't confirm whether the public key he received actually belongs to Alice.

30

Solution:

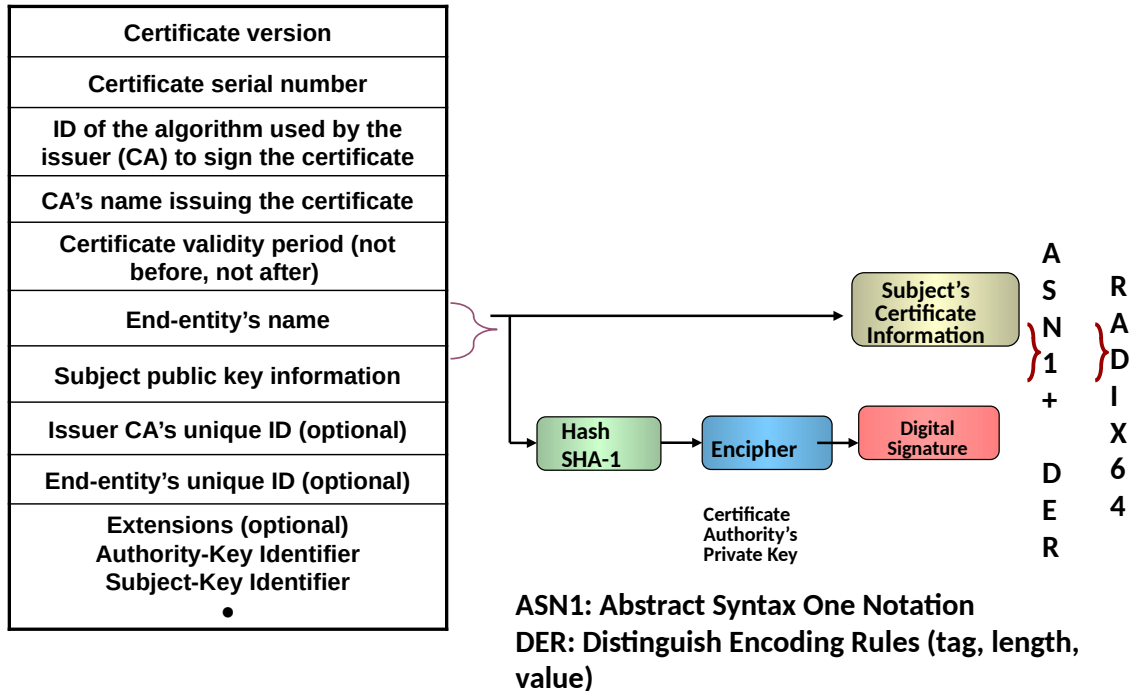
- Find a trusted entity: It is necessary to rely on a **trusted party** to verify the identity.
- Link an identity to a public key: It is recommended to associate an identity with a public key in a **certificate**.
- Certificate integrity: Thanks to the use of **digital signatures**, the certificate cannot be forged or tampered with.

31

Digital Certificates

- Digital certificates are electronic documents that verifiably link a **public key** to a specific entity, such as an individual, organisation or website. They are issued and digitally signed by a trusted third party, a **Certificate Authority (CA)**, which guarantees their validity and authenticity.
- Digital certificates can be thought of as digital identity cards.
- They allow for the **secure authentication** of the parties involved in online communications or transactions.
- A typical certificate contains:
 - The **public key** of the entity (e.g., a server, person, or device),
 - **Identifying information** about that entity,
 - The certificate's validity period (**start** and **expiration dates**).

X.509 Certificate



Types of Digital Certificates

- **Personal Certificate:** Used to authenticate an individual. It can be employed to digitally sign documents, emails, and other electronic communications.
- **Server Certificate:** Used to authenticate a server and establish a secure connection between the server and a client (e.g., in HTTPS communications).
- **Code-Signing Certificate (or Developer Certificate):** Used by software developers to prove that a piece of software originates from them and has not been altered or tampered with since it was signed.
- **IPsec Certificate:** Used to secure communications in Internet Protocol Security (IPsec) environments, typically for authenticating devices in virtual private networks (VPNs) or secure host-to-host communication.

Types of Digital Certificates (by Validation Level)

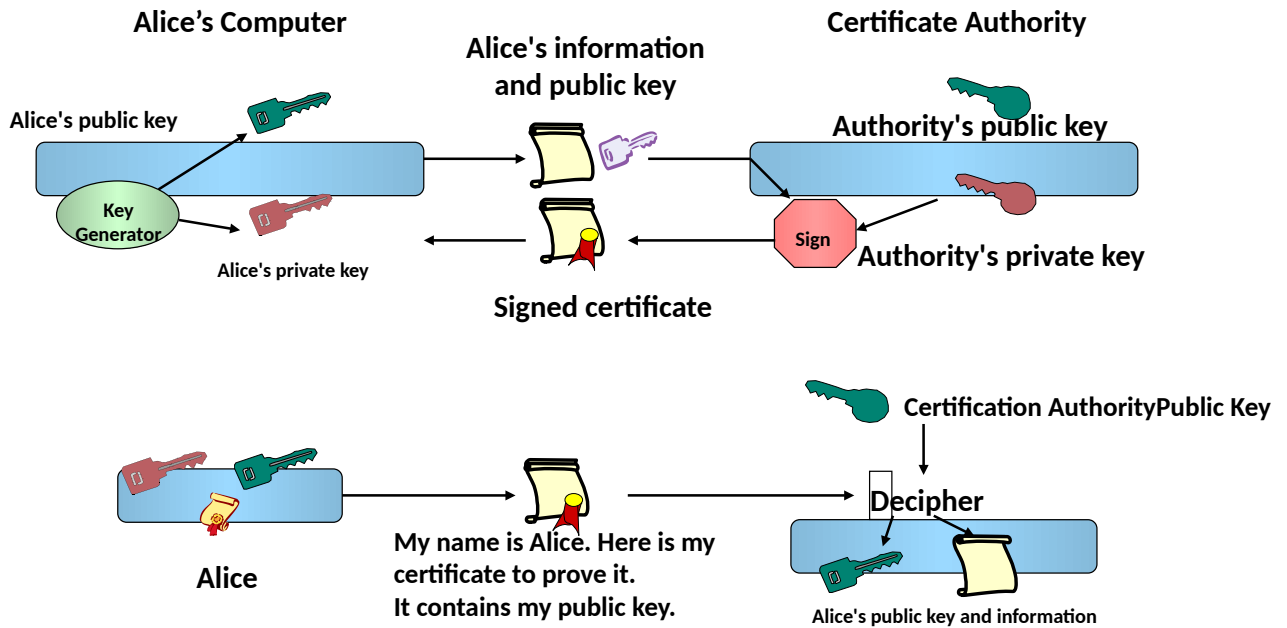
- **Domain-Validated (DV) Certificates:** DV certificates represent the most basic level of validation. They verify only that the applicant has control over the domain name associated with the certificate. These certificates are quick and easy to obtain but provide no information about the identity or legitimacy of the organization behind the website.
- **Organization-Validated (OV) Certificates:** OV certificates go beyond domain validation by also verifying the legal identity and existence of the organization that owns the domain. This additional layer of validation provides users with greater assurance when interacting with the website.
- **Extended Validation (EV) Certificates:** EV certificates offer the highest level of validation. In addition to the checks performed for OV certificates, the Certificate Authority (CA) conducts rigorous additional verification of the organization—such as confirming its operational status, physical address, and legal registration. Historically, EV certificates triggered a green address bar in many web browsers, signaling maximum trust to end users. (Note: As of recent browser updates, visual indicators like the green bar have been deprecated, though the validation process remains stringent.)

Chrome browser

DV/OV Certificate	 Secure https://www.microsoft.com/en-us/
EV Certificate	 PayPal, Inc. [US] https://www.paypal.com/us/home

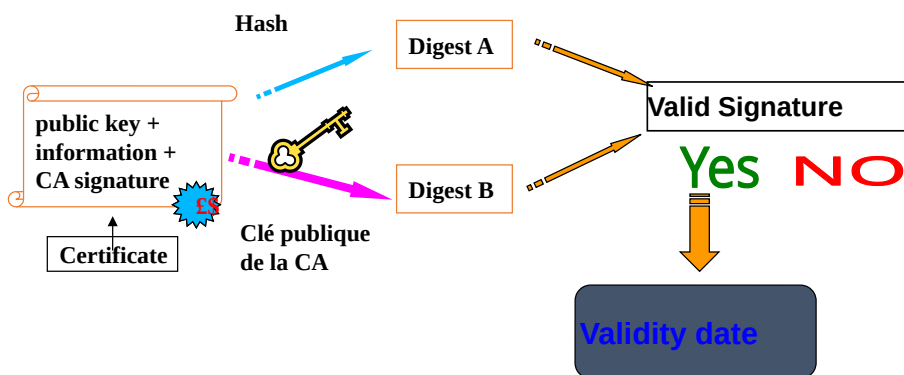
Firefox browser

DV/OV Certificate	 https://www.microsoft.com/en-us/
EV Certificate	 PayPal, Inc. (US) https://www.paypal.com/us/home



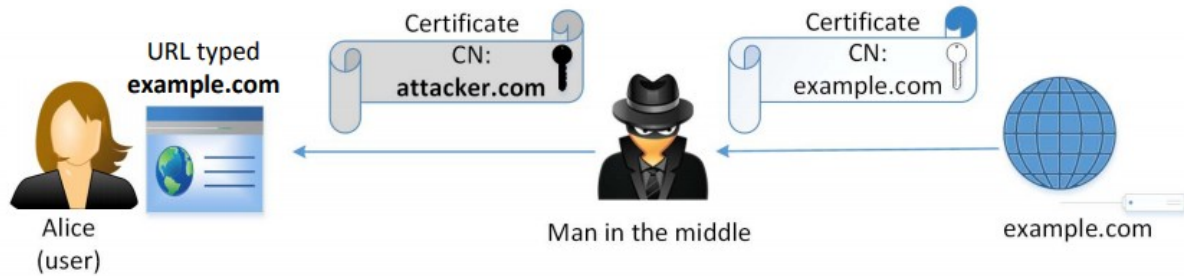
Who certifies the certificate authority's public key?

Certificat X.509



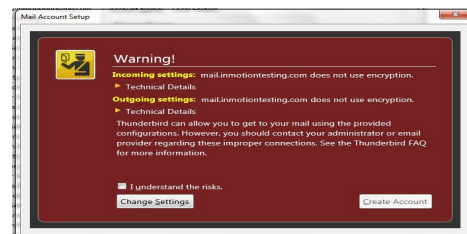
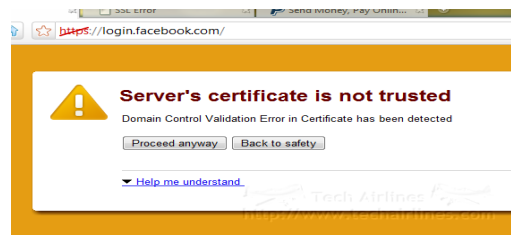
The certificate issued by the CA
The Certificate Revocation List (CRL) issued by the CA

Man-in-the-Middle (MITM) Attack and Certificate Validation

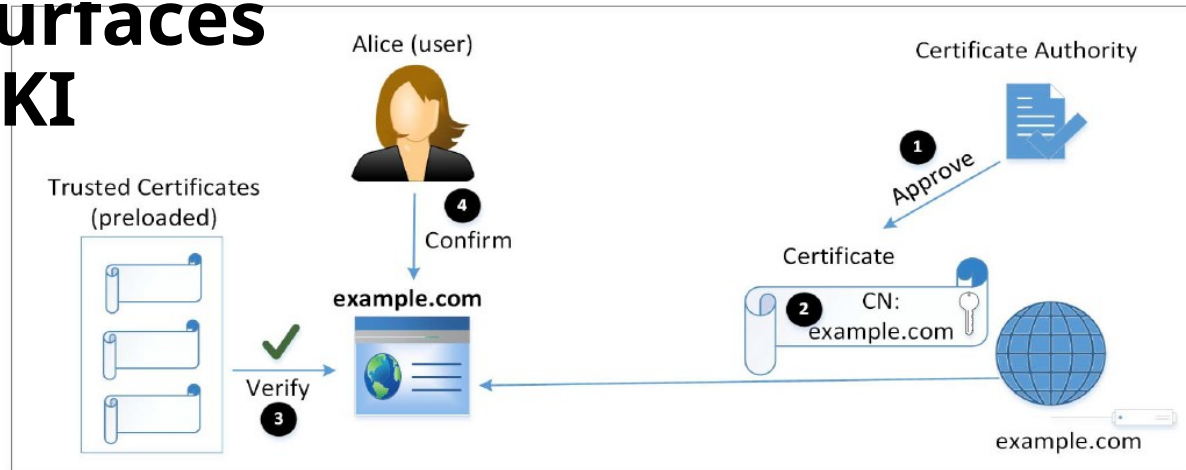


- The attacker's certificate is valid.
- The browser verifies whether the identity specified in the certificate's subject field matches Alice's intended identity.
- The certificate is issued for **attacker.com**
- But Alice is trying to reach **example.com**
- Since **attacker.com** $\neq$ **example.com**,
- The browser terminates the handshake protocol: MITM fails

39



Attacks Surfaces on PKI



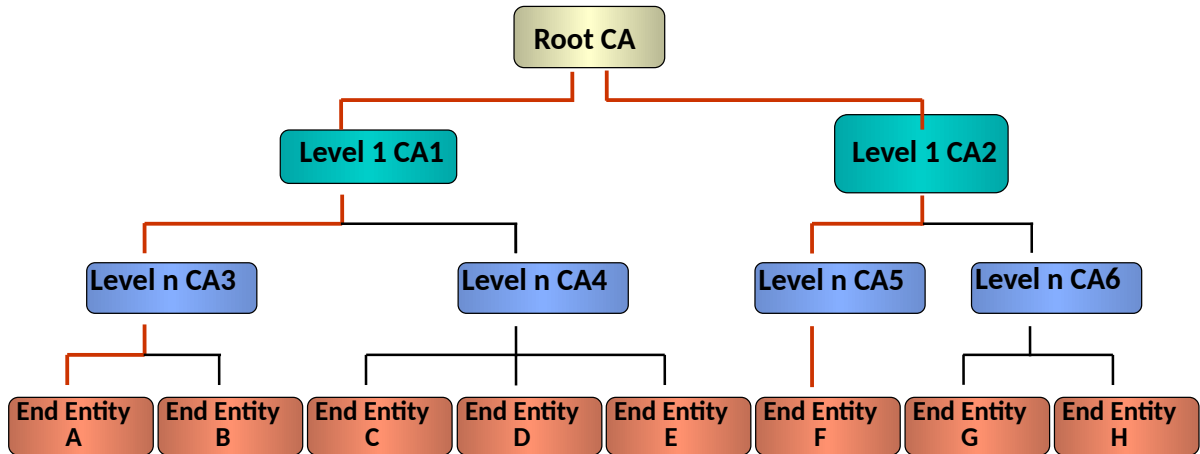
- 1- We can not trust what is said by a public key, we have to rely on a third-party CA to conduct necessary checks to ensure the correctness of the owner information.
- This is essential to defeat MITM attacks; client can know whether a received public key belongs to the intended server or not.
- 2- After CA verifies the identity, it will generate a certificate for the owner. Certificate should not be forged or tampered with. This is guaranteed by the one-way hash function and the digital signature algorithm.
- 3- Integrity of certificate protected by CA's signature. To verify signature, we need CA's public key. The public key (trusted certificate) must be preloaded by the client.
- 4- The client ensures that the subject in the certificate and user's intention matches.

41

Hierarchical Trust Model

- Having a single CA certify every public key is impractical;
 - Trust is established not through a single certificate, but via a **chain of trust**, a sequence of certificates linking an end-entity back to a trusted root Certificate Authority (CA).
 - **Example:**
 - A trusted **Root CA** issues a certificate to the **University of Chlef**, binding the university's identity to the public key of its Faculty of Exact Sciences (PK_{FSE}).
 - The Faculty of Exact Sciences (FSE) then acts as an **intermediate CA**.
 - FSE, in turn, issues a certificate to the **Computer Science Department**, binding its identity to its own public key (PK_{CSDep}).
- $$\text{sig}_{RootCA}(\text{Univ Chlef}, PK_{FSE}) \rightarrow \text{sig}_{FSE}(\text{CS Department}, PK_{CSDep})$$
- Root CA $\rightarrow$ University of Chlef $\rightarrow$ Faculty of Exact Sciences (FSE) $\rightarrow$ Computer Science Department
 - **What if the Root CA Is Compromised?** : If the root CA's private key is compromised, **the entire trust hierarchy collapses**: all certificates issued under that root can no longer be trusted.

Hierarchical Trust Model



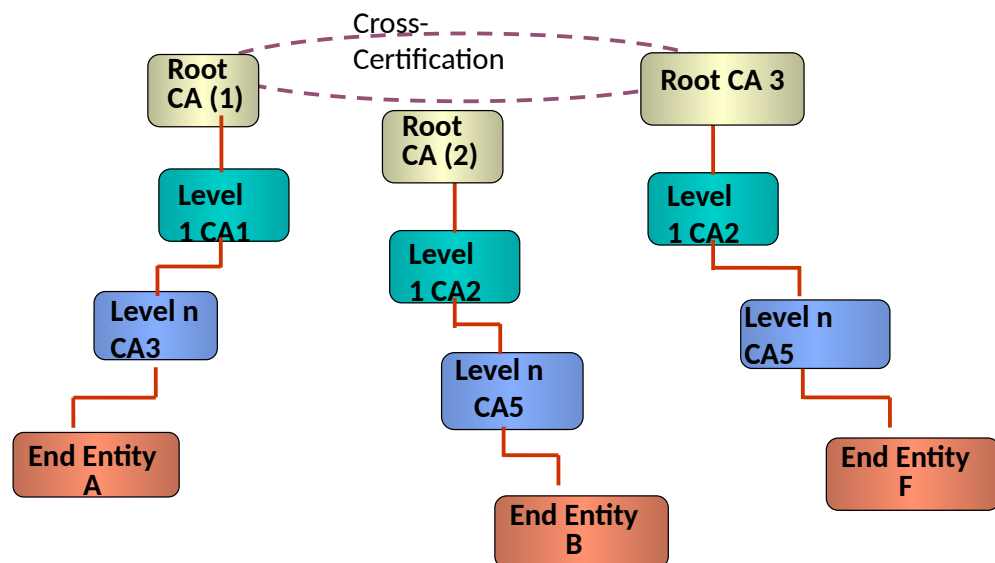
Advantage: Relatively simple to implement

Disadvantage: No cross-certification between CA.

If end entity A wants to certify end entity F, it must go all the way to CA5.

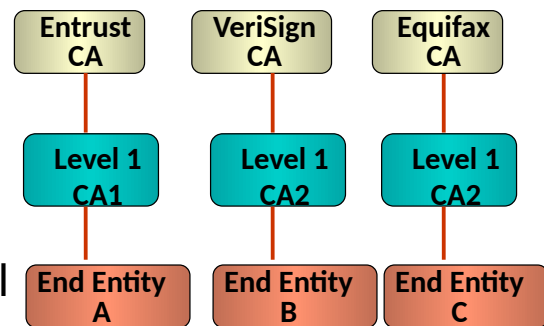
Mesh trust model

- In a **mesh trust model**, there is no hierarchy or centralized control.
- All Certification Authorities (CAs) recognize each other reciprocally.
- This model is generally adopted within federal or consortium-based organizations.
- It creates a distributed trust network in which each entity can validate certificates issued by other entities.



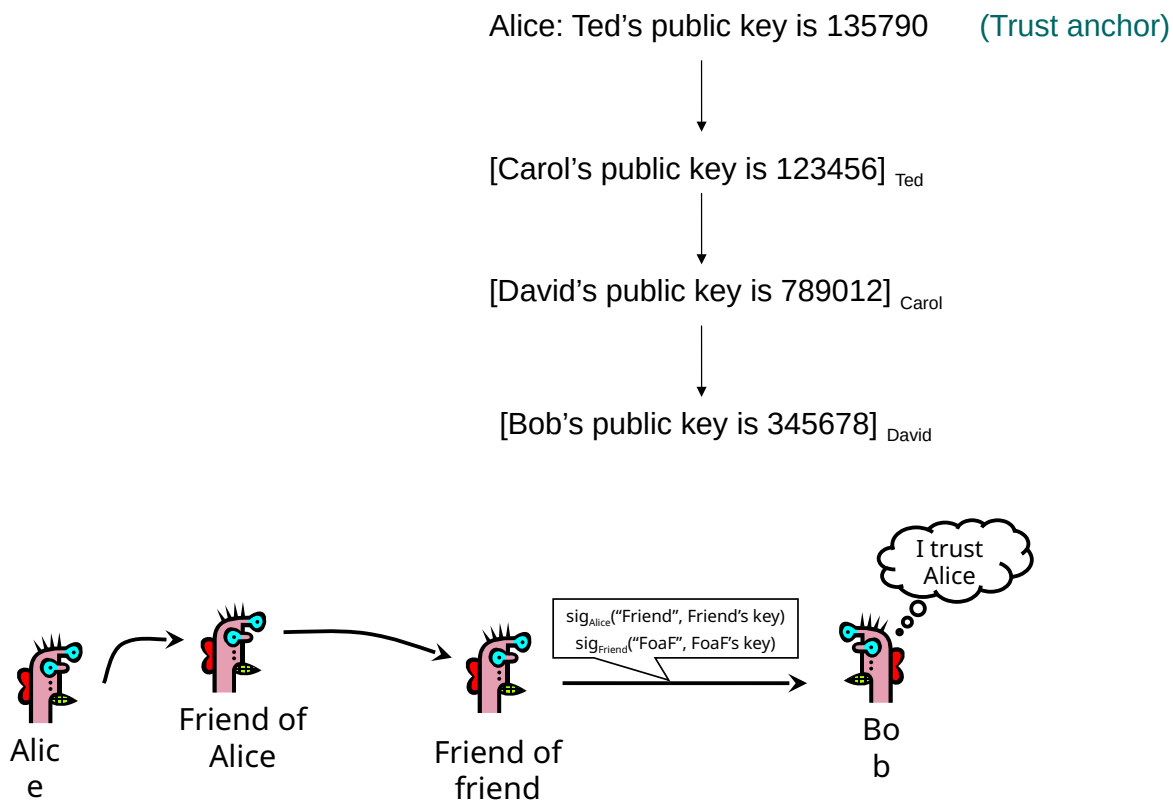
The Trust List Model (Web PKI)

- In the **trust list model**, also known as the Web PKI, a root **Certification Authority (CA)** is embedded directly into operating systems, web browsers, and other applications.
- Example: Modern web browsers are distributed with more than 100 preinstalled CA public keys.
- The root CA can issue certificates for websites, email addresses, code signing, and other digital identities.
- Users can trust certificates linked to this root CA because the trust anchor is already integrated into their software environment.



Web of Trust

- The Web of Trust is an alternative to traditional PKI in which trust is established not through a hierarchy of Certification Authorities (CAs), but through relationships among individual users or entities.
- This model is commonly linked to public-key encryption systems such as **Pretty Good Privacy (PGP)** and **GNU Privacy Guard (GPG)**.
- Each participant maintains a set of public keys they personally “**trust**”.
- If a **public-key** certificate is **signed** by one of these **trusted keys**, the contained public key is **considered valid**.
- Trust can be **transitive**: a key certified by a trusted entity may itself be used to certify additional keys.
- The Web of Trust creates a **decentralized**, user-driven trust network where validation depends on social or organizational relationships rather than a central authority.



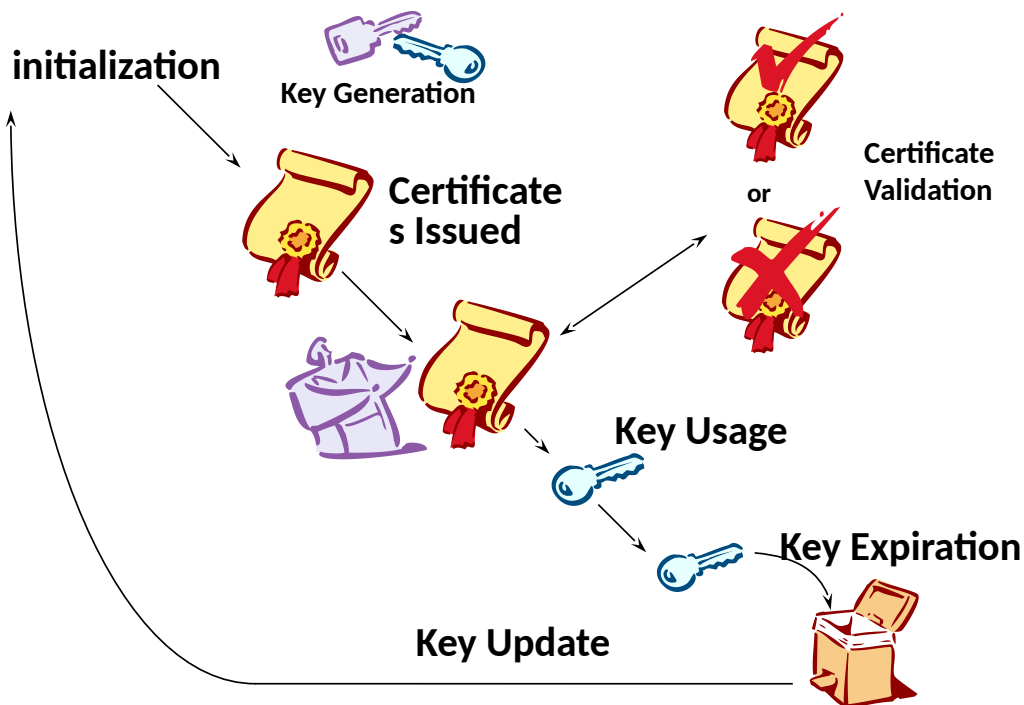
How to Verify a Self-Signed Certificate?

- Direct cryptographic verification is not possible. Instead, trust is based entirely on **how the certificate was obtained**:
- **Operating system**: If the certificate is bundled with the OS, trusting the operating system implies trusting the certificate.
- **Software package**: If the certificate is included with an application, trusting the software vendor implies trusting the certificate.
- **Manual installation**: If the user adds the certificate manually, trust rests on the user's own verification and judgment.
- **Untrusted sender**: If the certificate is received from an unknown or untrusted source, it must not be trusted.

Trusted Certificate Authorities in the Real World

- Not all trusted certificate authorities (CAs) are included in every browser or operating system trust store.
- A CA's inclusion depends on its compliance with **industry standards** and **approval** by platform vendors, such as Apple, Microsoft, Mozilla, and Google.
- According to **W3Techs data from April 2017**, the market share of SSL/TLS certificate issuers was dominated by:
 - **Comodo** (now **Sectigo**) : the largest issuer by volume,
 - Followed by **IdenTrust**,
 - The **Symantec CA group** (which included VeriSign, Thawte, and GeoTrust),
 - **GoDaddy**,
 - **GlobalSign**, and
 - **DigiCert**.

Key Lifecycle



Public Key Infrastructure (PKI)

- PKI is a comprehensive system that governs the creation, distribution, and revocation of public keys and digital certificates.
- It plays a crucial role in securing digital communications.
- PKI relies on multiple interconnected elements to provide encryption and digital signature services.
- **Asymmetric Encryption:**
 - Others send you encrypted messages using your **public key**.
 - You decrypt them with your corresponding **private key**.
- **Digital Signatures:**
 - You sign data with your **private key**.
 - Others verify the signature using your corresponding **public key**.

Core Components of a PKI

- **Certification Authority (CA):** A trusted entity responsible for issuing, revoking, and managing digital certificates. It validates the identity of certificate applicants and signs certificates to guarantee their authenticity.
- **Registration Authority (RA):** An entity that verifies the identity of certificate applicants before forwarding requests to the CA for issuance. It acts as the entry point for users seeking digital certificates.
- **End Entities (Users):** The entities, such as users, servers, or IoT devices that request digital certificates to secure their communications and online transactions.
- **Digital Certificates:** Electronic documents that bind a public key to a specific entity (e.g., a person, organization, or device). They are signed by the CA and used to verify identity and establish secure communications.

Core Components of a PKI

- **Certificate Revocation List (CRL):** A list published by the Certification Authority (CA) containing digital certificates that have been revoked before their normal expiration. It allows users to verify whether a certificate is still valid and take appropriate action.
- **Certification Policies and Certification Practice Statements (CP/CPS):** Certification policies define the rules and procedures the CA follows for issuing, revoking, and managing certificates. Certification Practice Statements (CPS) detail the specific operational practices the CA implements to ensure the security, reliability, and compliance of its services.
- **Key Management Infrastructure (KMI):** The infrastructure used to generate, store, and manage the cryptographic keys employed by the PKI, including the private keys of end entities and Certification Authorities.